Standardizing Data Staleness Telemetry: A Vendor-Neutral OpenTelemetry Semantic Convention for Data Freshness

Anirudh Rajmohan

July 20, 2026

Abstract

Data freshness—how recently a dataset reflects the real world—is a first-class reliability concern for analytics, machine learning, and operational decision systems. Yet despite decades of theory on information freshness (the *Age of Information*, AoI) and a thriving market of data-observability tools, there is no portable, vendor-neutral way to *emit* freshness as telemetry. OpenTelemetry (OTEL), now the de facto standard for observability signals, defines semantic conventions for database and messaging operations but none for data staleness; freshness today is re-implemented, incompatibly, inside each observability vendor and each data platform. We address this gap with three contributions. First, we propose a compact OpenTelemetry semantic convention for data staleness—a small set of metrics (`data.staleness.age`, `lag`, `last_update`, `records.behind`, and SLA threshold/breach signals) and attributes that unify freshness across SQL warehouses, streaming systems, batch pipelines, caches, NoSQL and lakehouse stores, and search/vector indexes under one vocabulary, grounded in the AoI model, including a *differential* (source-relative) form for indexes and replicas that trail their upstream. Second, we provide a reference implementation: a Python SDK that turns per-source *probes* into standardized metrics, a zero-configuration OpenTelemetry Collector receiver that scrapes SQL, streaming, object, and version-drift sources with no application code, and a Collector processor that centrally derives age and evaluates freshness SLAs. Third, we evaluate the implementation's correctness and overhead, measuring per-source emission cost of 22–39 μs in the SDK and sub-microsecond per-point processing in the Collector. We further validate the implementation end-to-end against real backends—PostgreSQL, Kafka, Redis, Kinesis, and a schema registry—on a cloud instance, asserting that emitted metrics are numerically correct (age equal to a known injected offset, exact consumer lag) and that failures are made visible rather than fabricated, not merely that metrics appear. Our aim is not a new freshness metric but the *consolidation* of a well-studied one into the dominant observability framework, so that freshness becomes as portable and queryable as latency or error rate. The artifacts are released as open source under Apache-2.0.

1 Introduction

Modern organizations make decisions on derived data: dashboards, feature stores, recommendation indexes, fraud models, and financial reports all consume data that has traveled through ingestion, transformation, and serving layers. When that pipeline stalls, the data does not become *wrong* so much as *old*: a dashboard silently shows yesterday's numbers, a model scores on features that no longer reflect reality. This failure mode—*staleness*— is distinct from the failures that traditional observability captures well. A pipeline can have zero errors and low latency on every individual operation and still serve hours-old data because an upstream producer stopped.

Two largely separate communities have studied this. In networked and cyber-physical systems, the *Age of Information* (AoI) line of work formalizes freshness as the time since the most recently received

update was generated at its source [1, 2], with a rich body of analysis and variants such as Peak AoI and Age of Synchronization [2]. In data engineering, a generation of *data-observability* platforms [4, 5, 6] and tools such as dbt `source freshness` [7] and Kafka lag monitors like Burrow [8] measure freshness operationally. Both agree on the core quantity. Neither produces it as portable telemetry.

The consequence is fragmentation. Each data-observability vendor models and stores freshness differently; freshness computed by dbt cannot be compared, in one query, with freshness of a Kafka topic or a Redis cache; and alerting on freshness is locked to whichever proprietary system computed it. Meanwhile OpenTelemetry (OTEL) has become the cross-vendor standard for emitting metrics, traces, and logs, with *semantic conventions* that make signals comparable across systems [9]. OTEL defines conventions for database client metrics and messaging/Kafka metrics, but these capture operation *duration* and message/row *counts*—not how old the data is. As of the 1.42 conventions and the published 2026 semantic-conventions roadmap, there is no data-freshness metric, and not even a standardized consumer-lag metric [10, 11].

This paper closes that gap. We argue that freshness deserves the same treatment latency received: a small, agreed-upon convention plus reference instrumentation, so any producer can emit it and any backend can consume it. Concretely, we contribute:

1. **A semantic convention for data staleness** (Section 3): metric names, units, instrument types, and attributes that express AoI-style freshness uniformly across SQL/warehouse, streaming, batch-pipeline, and cache/object-store sources.
2. **A reference implementation** (Sections 4–5): an Apache-2.0 Python SDK (`otel-staleness`) built on `opentelemetry-python`, and a Go OpenTelemetry Collector processor (`datastaleness`) that derives age and evaluates SLAs centrally.
3. **An evaluation** (Section 6) of correctness and runtime overhead, with reproducible microbenchmarks.

We are explicit about novelty: the *metric* of staleness is not new—it is AoI, and it is what data-observability vendors already compute. Our contribution is *standardization and consolidation*: expressing that known quantity once, in OpenTelemetry, with a portable implementation.

2 Background and Related Work

Age of Information. AoI was introduced to capture freshness in status-update systems [1] and has since become a canonical metric for information freshness in communication, IoT, and control systems [2]. For a source whose freshest delivered update was generated at time t_e, the age at wall-clock time t is $\Delta(t) = t - t_e$: a sawtooth that rises linearly until a newer update arrives and resets it. Our `data.staleness.age` is exactly this quantity, applied to data systems rather than wireless links. AoI theory also motivates our distinction between *age* (which grows during a stall) and one-shot *lag* (the delay a record incurs traversing the pipeline).

Data freshness in data management. The data-management community studied freshness and timeliness well before "data observability" as a term, e.g. Peralta's survey of data freshness and accuracy in integration systems [3]. Freshness is now treated as a pillar of data observability alongside volume, schema, distribution, and lineage [4]. Practitioner definitions converge on the lag between when an event occurred and when it is queryable [5, 6].

Operational tooling. dbt computes source freshness from a `loaded_at` column and warns/errors against configured thresholds [7]; Kafka ecosystems track consumer lag with tools such as Burrow [8]; commercial platforms (Monte Carlo, Metaplane, Sifflet) provide freshness monitoring as a managed service [4, 5, 6]. Each is valuable and each is a silo: the freshness signal lives inside the tool that produced it.

Table 1: Where existing approaches stand. "Portable emission" means the freshness signal is produced as open, cross-vendor telemetry.

Approach	Formal model	Multi-source	Portable emission
AoI theory [1, 2]	✓	partial	—
dbt source freshness [7]	partial	— (batch only)	—
Kafka lag tools [8]	partial	— (streaming only)	—
Data-observability SaaS [4, 5]	partial	✓	—
OTEL DB/messaging semconv [10]	—	partial	✓
This work	✓	✓	✓

OpenTelemetry. OTEL standardizes telemetry emission and, via semantic conventions, the *meaning* of that telemetry [9]. Conventions exist for database and messaging metrics [10], and the data model includes a low-level Prometheus-style "staleness marker" for missing timeseries—an unrelated concept to data freshness. No convention emits data age. Our work proposes one and is structured to serve as the basis for an OpenTelemetry Enhancement Proposal (OTEP).

Positioning. Table 1 summarizes how existing approaches relate to the three properties we care about: a formal freshness model, operational measurement across heterogeneous sources, and portable/vendor-neutral emission.

3 The Data Staleness Convention

3.1 Model

Let a *logical source* be any dataset whose freshness we care about: a table, a topic, a dbt model, a cache namespace, or an object-store prefix. For the freshest record of a source with event time t_e, observed/processed at t_p, with current wall-clock time t_{now}:

$$\text{lag} = t_p - t_e, \tag{1}$$

$$\text{age} = t_{now} - t_e \quad (\geq \text{lag}). \tag{2}$$

`lag` is the delay a record incurred moving through the pipeline; it is fixed per record. `age` keeps increasing until a newer record arrives. If a source stops producing, lag of the last record is constant while age rises without bound—precisely the condition freshness monitoring exists to detect.

3.2 Metrics

All durations use the UCUM unit `s`; timestamps are Unix epoch seconds. Point-in-time values are reported as asynchronous (observable) gauges; counts of events use monotonic counters. Table 2 lists the convention.

3.3 Attributes

A small attribute set makes points comparable and sliceable: `data.source.system` (e.g. `postgresql`, `kafka`, `dbt`, `redis`, `s3`; reusing `db.system`/`messaging.system` values where they exist), `data.source.name` (table/topic/model/prefix), `data.source.namespace` (schema/consumer group/bucket), `data.staleness.metho` (how the value was derived; see below), `data.staleness.partition`, and `data.pipeline.stage` (`ingest`/`transform`/`serve`).

Table 2: Proposed `data.staleness.*` metrics.

Metric	Instrument	Unit	Meaning
`data.staleness.age`	Gauge	`s`	$t_{now} - t_e$; primary AoI metric.
`data.staleness.lag`	Gauge	`s`	$t_p - t_e$ of most recent record.
`data.staleness.last_update.timestamp`	Gauge	`s`	Unix time of last update.
`data.staleness.records.behind`	Gauge	{`record`}	Positional backlog (e.g. Kafka lag).
`data.staleness.sla.threshold`	Gauge	`s`	Max acceptable age (SLO target).
`data.staleness.sla.breached`	Gauge	`1`	1 if age $>$ threshold else 0.
`data.staleness.sla.breaches`	Counter	{`breach`}	Transitions into breach.

The `method` attribute records the measurement recipe and makes heterogeneous sources self-describing: `max_timestamp` (SQL `MAX(updated_at)`), `watermark`, `consumer_lag` (streaming), `run_completion` (batch jobs), `object_mtime` (object storage), `ttl_age` (caches), and `heartbeat`.

3.4 Per-source mapping

The convention is useful only if each source type has an unambiguous recipe. SQL/warehouse sources set $age = t_{now} - \texttt{MAX}(\textit{ts column})$. Streaming sources use the event timestamp of the last consumed record for age and lag, and offset lag for `records.behind`, reported per partition. Batch/dbt/Airflow sources set $age = t_{now} - \textit{last successful run end}$. Caches and object stores use last write / last-modified time. These mirror what existing tools already compute internally; the convention's value is that they now emit *the same metric*. Because the model rests only on a source event time, it extends to further families with no new metrics—NoSQL and graph stores (e.g. Cassandra `WRITETIME()`), time-series databases, lakehouse table formats (Iceberg/Delta/Hudi snapshot commit time), and external feeds—which become new `data.source.system` / `method` values rather than new instruments.

Differential (source-relative) freshness. Some systems' freshness is best expressed against an *upstream* rather than against t_{now}: a search or vector index that trails its source corpus, or a replica/CDC target that trails its primary. We capture this with the existing `lag` metric, setting the record's event time to the upstream write/commit time and naming the upstream in a `data.staleness.relative_to` attribute (methods `index_lag`, `replication_lag`). Using source time on both sides makes the measurement robust to clock skew between the two systems. Absolute (`age` vs t_{now}) and differential (`lag` vs upstream) freshness are complementary and may be emitted together; index staleness in retrieval-augmented generation pipelines is a prominent case where the index can be behind its corpus while also being old in absolute terms.

4 System Design

The reference implementation has two cooperating halves (described below): a client-side SDK that knows how to *measure* each source, and a Collector-side processor that can *derive and evaluate* freshness centrally.

Architecture (textual). *Producers* expose freshness via *probes*. A probe reports the event time of the freshest record for one logical source. The `StalenessMonitor` registers the standardized observable gauges once and, on each collection, pulls readings from all probes, computes age, and emits observations. Telemetry flows over OTLP to an OpenTelemetry Collector, where the `datastaleness` processor can (a) derive `data.staleness.age` for producers that emit only `last_update.timestamp`, and (b) attach `data.staleness.sla.threshold`/`breached` according to operator policy. Any OTEL-compatible backend (Prometheus, OTLP store, vendor) then stores and queries the result.

Why two halves. Some producers can compute age themselves (a service that knows its event times); others are best instrumented with the bare minimum—"here is the timestamp of my newest row"—and should not embed clock logic or SLA policy. Centralizing derivation and SLA evaluation in the Collector keeps thresholds in one operational place and lets the same age computation apply uniformly, which also avoids clock-skew logic sprawling across services.

Design principles. (1) *Minimum producer burden*: a probe is a plain callable returning a timestamp. (2) *Fault isolation*: a failing probe must never break collection of the others. (3) *No new transport*: everything rides standard OTEL metrics. (4) *Policy where it belongs*: SLAs are operator configuration in the Collector, not hard-coded in applications.

5 Implementation

5.1 Python SDK

`otel-staleness` is built on `opentelemetry-api/sdk`. The core is a `FreshnessReading` dataclass and a `StalenessMonitor` that registers the convention's instruments via observable-gauge callbacks. Probes ship for the four core families (SQL/warehouse, streaming, batch, cache/object store) plus two *differential* probes—`IndexFreshnessProbe` and `ReplicationFreshnessProbe`—that report how far a search/vector index or a replica trails its upstream via `lag` and `relative_to`:

```
from otel_staleness import StalenessMonitor, conventions as sc
from otel_staleness.probes import SQLFreshnessProbe

monitor = StalenessMonitor()
monitor.add_probe(SQLFreshnessProbe(
    fetch_max_epoch=lambda: latest_updated_at(),  # Unix s or datetime
    source_name="orders", system=sc.System.POSTGRESQL,
    namespace="public", sla_threshold_seconds=300))
monitor.start()
```

Every probe accepts a plain callable, so probes are unit-testable without a live backend; `SQLFreshnessProbe` additionally offers a `from_sqlalchemy` constructor. The monitor guards each probe call so a raised exception is swallowed and the remaining probes still report—design principle (2).

5.2 Collector processor

`datastaleness` is a Go metrics processor (Collector API `v0.110.0`, pdata `v1.16.0`). On each batch it: scans for existing `age` points; optionally derives `age` from `last_update.timestamp` as $\max(0, t_{now} - ts)$; and, for each age point, looks up the first matching SLA rule (selectors on `data.source.system`/`nam` with a default threshold) to emit `threshold` and `breached` points. Configuration:

```
processors:
  datastaleness:
    compute_age_from_last_update: true
    evaluate_sla: true
    default_threshold: 5m
    slas:
      - source_system: kafka
        threshold: 30s
      - source_name: orders
        threshold: 1m
```

Table 3: Python SDK: collection + in-memory export cost.

Sources	Cycle time (ms)	Per source (μs)
100	2.16	21.7
1,000	25.7	25.7
10,000	385.3	38.5

6 Evaluation

We evaluate (i) functional correctness and (ii) runtime overhead. Experiments ran in a containerized Linux environment (Python 3.10, Go 1.25, 2 vCPU).

6.1 Correctness

Both components ship with unit tests asserting the convention's semantics: age derived from last-update time; explicit age preferred when given; age clamped to be non-negative under clock skew; SLA breach edges; per-partition Kafka lag; method/attribute tagging; and probe fault isolation. The Python suite (17 tests) and Go suite (7 tests) pass. Representative invariants checked: $age = t_{now} - t_e$; $age \geq 0$; and `breached` $= \mathbb{1}[age > \textit{threshold}]$.

6.2 Overhead: SDK

We measured the cost of one collection cycle (callback execution plus in-memory export) as a function of the number of monitored sources, averaged over 20 iterations. Table 3 shows the SDK adds tens of microseconds per source. Even at 10,000 sources a full collection completes in ~ 385 ms; at a typical 15–60 s metric export interval this is negligible. Per-source cost is dominated by attribute-set construction and observation allocation, not by the freshness arithmetic.

6.3 Overhead: Collector processor

We benchmarked `processMetrics` on a batch of 1,000 `last_update.timestamp` points with both age derivation and SLA evaluation enabled (Go `testing` benchmark). It runs in ~ 0.93 ms per batch ($\approx 0.93\,\mu s$ per point) with ~ 1.5 MB allocated per batch. Throughput therefore exceeds one million points per second per core, far above the point rates produced by freshness telemetry, which is emitted per source per export interval rather than per request.

6.4 Discussion of results

The measured overheads confirm the design intent: freshness is a low-cardinality, low-frequency signal (one set of points per source per export), so even naive implementations are comfortably cheap. The interesting cost in practice is cardinality—number of distinct `(system, name, partition)` tuples—which operators already manage for other metrics and which the attribute set is deliberately kept small to bound.

7 Limitations and Future Work

Our evaluation establishes correctness and overhead, not end-to-end detection quality on production workloads; a natural next step is a field study comparing breach-detection latency against an existing data-

observability tool on the same pipelines. Clock skew between producer and Collector affects derived age; we clamp to non-negative but do not yet correct skew (e.g. via NTP offset attributes). The convention intentionally omits richer freshness variants from AoI theory (Peak AoI, Age of Incorrect Information); these could be added as optional metrics. Finally, the names here are a Development-stage proposal: the intended path to adoption is an OTEP and community review, during which names may change. Additional first-party probes (BigQuery, Snowflake information-schema, Kinesis, GCS) and a Collector *receiver* that actively polls sources are straightforward extensions.

8 Conclusion

Freshness is a known and well-studied quantity that, paradoxically, has no standard way to be *emitted*. We proposed a compact OpenTelemetry semantic convention for data staleness that unifies AoI theory with the fragmented practice of data observability, and we released a reference implementation—a Python SDK and a Collector processor—that is correct and cheap enough to run everywhere. By making data age a portable, queryable telemetry signal alongside latency and errors, we hope to let teams reason about "how old is my data" with the same rigor they already apply to "how fast" and "how broken."

Artifact Availability

The semantic-convention specification, Python SDK (`otel-staleness`), Go Collector processor (`datastaleness`), demo deployment, and benchmarks are released as open source under Apache-2.0.

References

[1] S. Kaul, R. Yates, and M. Gruteser. "Real-time status: How often should one update?" In *IEEE INFOCOM*, 2012.

[2] R. D. Yates et al. "Age of Information: An Introduction and Survey." *IEEE J. Sel. Areas Commun.*, 2021. arXiv:2007.08564.

[3] V. Peralta. "Data Freshness and Data Accuracy: A State of the Art." Technical Report, Universidad de la República, 2006.

[4] Monte Carlo Data. "Data Freshness Explained." https://www.montecarlodata.com/blog-data-freshness-explained/.

[5] Metaplane. "What is data freshness? Definition, examples, and best practices." https://www.metaplane.dev/blog/data-freshness-definition-examples.

[6] Sifflet. "What Is Data Freshness in Data Observability?" https://www.siffletdata.com/blog/data-freshness.

[7] dbt Labs. "Source freshness." dbt documentation. https://docs.getdbt.com/docs/build/sources.

[8] LinkedIn. "Burrow: Kafka Consumer Lag Checking." https://github.com/linkedin/Burrow.

[9] OpenTelemetry. "Semantic Conventions." https://opentelemetry.io/docs/concepts/semantic-conventions/.

[10] OpenTelemetry. "Semantic conventions for messaging metrics." https://opentelemetry.io/docs/specs/semconv/messaging/messaging-metrics/.

[11] OpenTelemetry. "Semantic Conventions 2026 Roadmap." GitHub issue, open-telemetry/semantic-conventions #3330.